SQL MINI PROJECT

RETAIL STORE



Data Analysis using SQL

Prepared By: AJAY KUMAR

Project: SQL Mini Project – Retail Store

Database: Retail Store Database

MySQL Database: mini_project

Tool: MySQL Workbench

SQL • DATABASE • DATA ANALYSIS

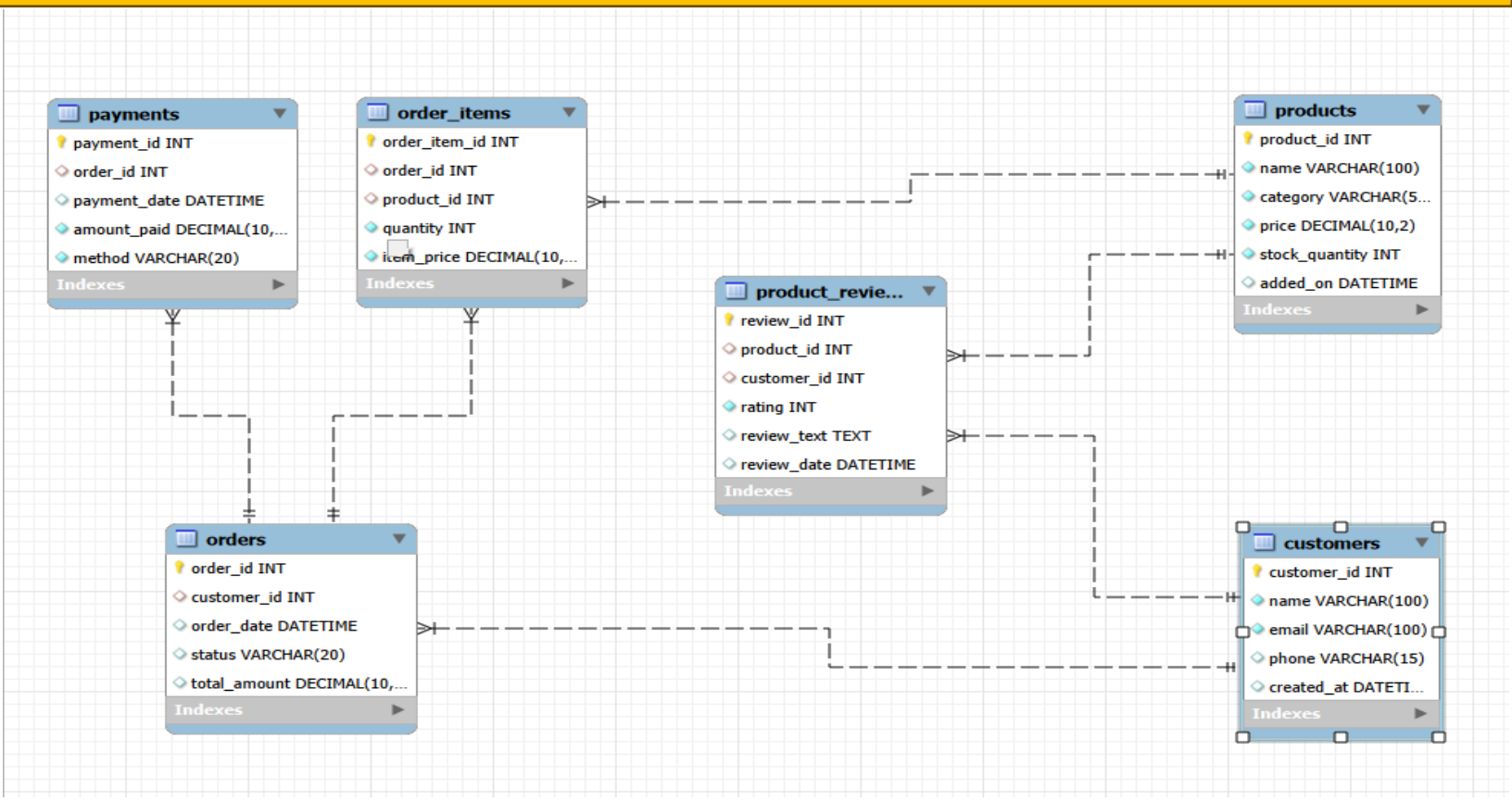
FROM Data to Insights with SQL

OBJECTIVE

The objective of this project is to analyze a retail store database using MySQL by retrieving and manipulating data from multiple related tables.

The project focuses on applying SQL concepts such as basic queries, filtering, formatting, aggregate functions, multi-table joins, subqueries, and set operations to derive meaningful information from customer, order, product, payment, and review data.

RETAIL STORE DATABASE



DATABASE SETUP — DDL SCRIPTS

```
create database mini_project;

use mini_project;
```

Customers

```
CREATE TABLE customers (
   customer_id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100) NOT NULL,
   email VARCHAR(100) NOT NULL UNIQUE,
   phone VARCHAR(15),
   created_at DATETIME DEFAULT CURRENT_TIMESTAMP );
```

Products

```
CREATE TABLE products (
   product_id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100) NOT NULL,
   category VARCHAR(50) NOT NULL,
   price DECIMAL(10,2) NOT NULL,
   stock_quantity INT NOT NULL DEFAULT 0,
   added_on DATETIME DEFAULT CURRENT_TIMESTAMP );
```

Orders

```
CREATE TABLE orders (
  order_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT,
  order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
  status VARCHAR(20) DEFAULT 'Pending',
  total_amount DECIMAL(10,2),
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id) );
```

Order Items

```
CREATE TABLE order_items (
  order_item_id INT PRIMARY KEY AUTO_INCREMENT,
  order_id INT,
  product_id INT,
  quantity INT NOT NULL CHECK (quantity > 0),
  item_price DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id) );
```

Payments

```
CREATE TABLE payments (
  payment_id INT PRIMARY KEY AUTO_INCREMENT,
  order_id INT,
  payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
  amount_paid DECIMAL(10,2) NOT NULL CHECK (amount_paid > 0),
  method VARCHAR(20) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(order_id) );
```

Product Reviews

```
CREATE TABLE product_reviews (
review_id INT PRIMARY KEY AUTO_INCREMENT,
product_id INT,
customer_id INT,
rating INT NOT NULL,
review_text TEXT,
review_date DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (product_id) REFERENCES products(product_id),
FOREIGN KEY (customer_id) REFERENCES customers(customer_id) );
```

Question Modes

EASY: BASIC SELECT, FILTERING, FORMATTING, AGGREGATE FUNCTIONS .

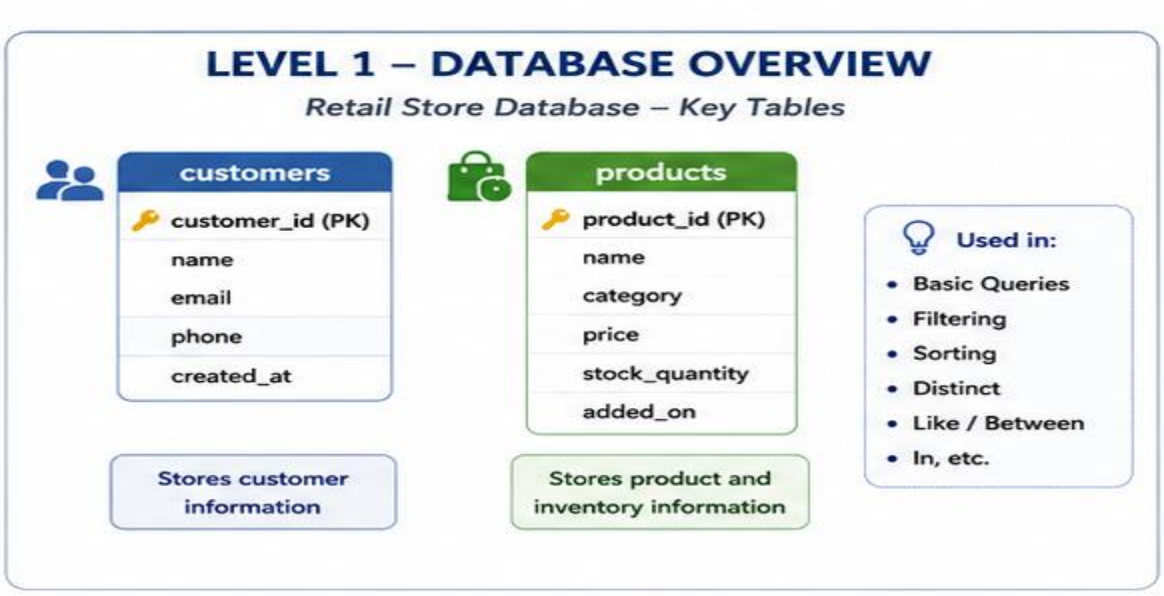
MODERATE: GROUP BY, JOINS, UNION/SET OPERATIONS .

HARD: SUBQUERIES, CORRELATED SUBQUERIES, CUSTOMER-SPECIFIC ANALYSIS.



Table	Purpose
Customers	Stores customer Information
Products	Stores product and Inventory Information
Orders	Stores order-level Information
Order_Items	Stores products Included in each order
Payments	Stores payment transactions
Product_Reviews	Stores customer product ratings and reviews

Level 1: Basics



Q1. Retrieve customer names and emails for email marketing.

Input: SELECT name, email FROM customers;

Output:

name	email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com
Leslie Wilson	user13@example.com
Deborah Arias	user14@example.com
Austin Flores	user15@example.com
Amy Landry	user16@example.com
Randy Mooney	user17@example.com

customers 1 ×

Q2. View complete product catalog with all available details.

Input: SELECT * FROM products;

Output:

product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
14	Listen Development	Home	296.17	132	2024-10-12 11:49:26
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55

products 2 ×

Q3. List all unique product categories.

Input: SELECT DISTINCT category FROM products;

Output:

category
Home
Clothing
Electronics
Toys
Books

Q4. Show all products priced above ₹1,000.

Input: SELECT * FROM products WHERE price > 1000;

Output:

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30

products 4

Q5. Display products within a mid-range price bracket (₹2,000 to ₹5,000).

Input: SELECT * FROM products WHERE price BETWEEN 2000 AND 5000;

Output:

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13
24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17

products 5 ×

Q6. Fetch data for specific customer IDs (e.g., From loyalty program list).

Input: SELECT * FROM customers WHERE customer_id IN (1, 4, 8);

(1, 4, and 8 are selected sample customer IDs for demonstrating the IN operator.)

Output:

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
NULL	NULL	NULL	NULL	NULL

Q7. Identify customers whose names start with the letter 'A'.

Input: SELECT * FROM customers WHERE name LIKE 'A%';

Output:

customer_id	name	email	phone	created_at
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
NULL	NULL	NULL	NULL	NULL

Q8. List electronics products priced under ₹3,000.

Input: SELECT name FROM products WHERE category = 'Electronics' AND price < 3000;

Output:

name
Available Answer
If Whatever
Place Low
Series Page
Despite Win
Actually Term
Southern Thing

Q9. Display product names and prices In descending order of price.

Input: SELECT name, price FROM products ORDER BY price DESC;

Output:



name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66
Involve Officer	4244.09
Group But	4180.23
Every Amount	4173.60
Move Small	4168.08
Drop Remember	4160.45
Data Add	4063.38
Action Ask	4017.01

products 11 ×

Q10. Display product names and prices, sorted by price and then by name.

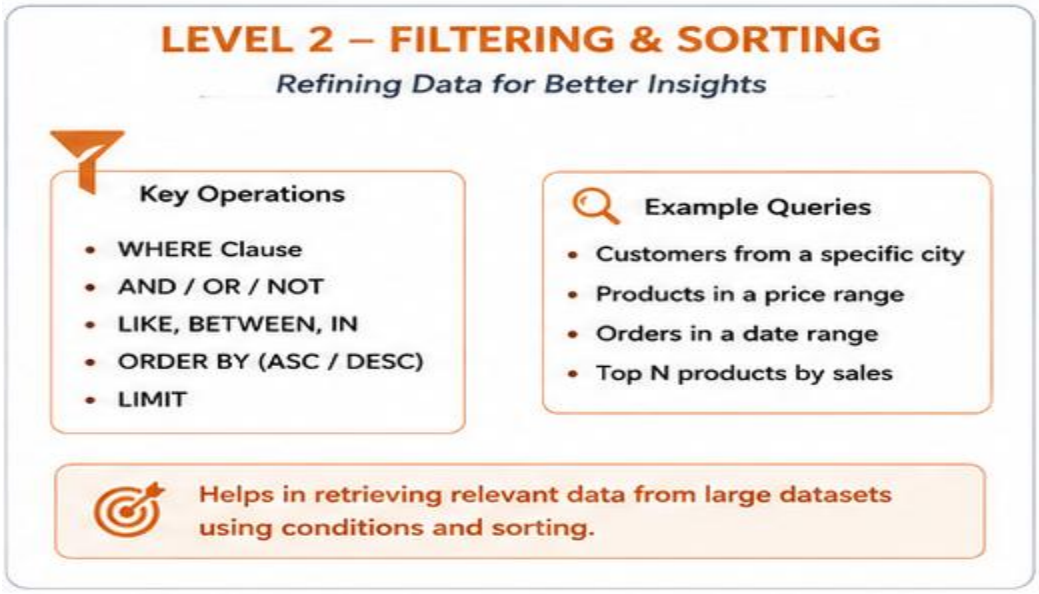
Input: SELECT name, price FROM products ORDER BY price DESC, name ASC;

Output:

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66
Involve Officer	4244.09
Group But	4180.23
Every Amount	4173.60
Move Small	4168.08
Drop Remember	4160.45
Data Add	4063.38
Action Ask	4017.01

products 10 ×

Level 2: Filtering and Formatting



Q1. Retrieve orders where customer information is missing (possibly due to data migration or deletion).

Input: SELECT * FROM orders WHERE customer_id is NULL;

Output:

No orders with missing customer information were found.

order_id	customer_id	order_date	status	total_amount
NULL	NULL	NULL	NULL	NULL

Q2. Display customer names and emails using column aliases for frontend readability.

Input: SELECT name as customer_name, email as email_id FROM customers;

Output:

customer_name	email_id
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com
Leslie Wilson	user13@example.com
Deborah Arias	user14@example.com
Austin Flores	user15@example.com
Amy Landry	user16@example.com
Randy Mooney	user17@example.com

Q3. Calculate total value per item ordered by multiplying quantity and item price.

Input: SELECT * , (quantity*item_price) as total_value FROM order_items;

Output:

order_item_id	order_id	product_id	quantity	item_price	total_value
1	1	19	2	4707.14	9414.28
2	2	6	3	177.40	532.20
3	3	9	3	985.19	2955.57
4	3	23	1	2208.99	2208.99
5	4	35	2	4734.89	9469.78
6	5	19	1	4707.14	4707.14
7	5	7	2	4897.36	9794.72
8	6	7	3	4897.36	14692.08
9	6	37	1	1429.45	1429.45
10	6	15	1	723.97	723.97
11	6	35	3	4734.89	14204.67
12	7	30	1	3043.67	3043.67
13	8	10	3	4801.78	14405.34
14	8	5	3	4722.66	14167.98
15	8	31	2	2070.37	4140.74
16	9	7	3	4897.36	14692.08
17	9	44	1	396.11	396.11

Q4. Combine customer name and phone number in a single column.

Input: SELECT CONCAT(name, ' ' ,phone) as name_phone FROM customers;

Output:

name_phone
Thomas Owens 142-479-1945
Charles Grant 9153947511
Kaitlin Richards 2073473421
Christina Williams 586-605-5061x06
David Allen (751)456-8289x1
Mark Duke (144)957-2811
Briana Wright 223-833-9635
John Bryan 045.568.0798x27
Jason Thompson 1862659420
Shawn Hill (268)113-3152x7
Walter Jenkins 536-329-0817x71
Mary Knight 361-636-3802
Leslie Wilson +1-256-261-1984
Deborah Arias 811-821-2144x97
Austin Flores 329.901.1576x66
Amy Landry +1-278-019-3748
Randy Mooney (158)927-7313x3

Q5. Extract only the date part from order timestamps for date-wise reporting.

Input: SELECT DATE(order_date) as order_day FROM orders;

Output:

order_day
2025-03-02
2024-10-09
2025-05-08
2024-09-19
2025-04-08
2024-10-25
2024-07-29
2024-07-30
2025-06-10
2025-02-16
2025-03-11
2025-01-15
2025-05-12
2024-09-24
2024-07-18
2024-09-09
2024-08-19

Q6. List products that do not have any stock left.

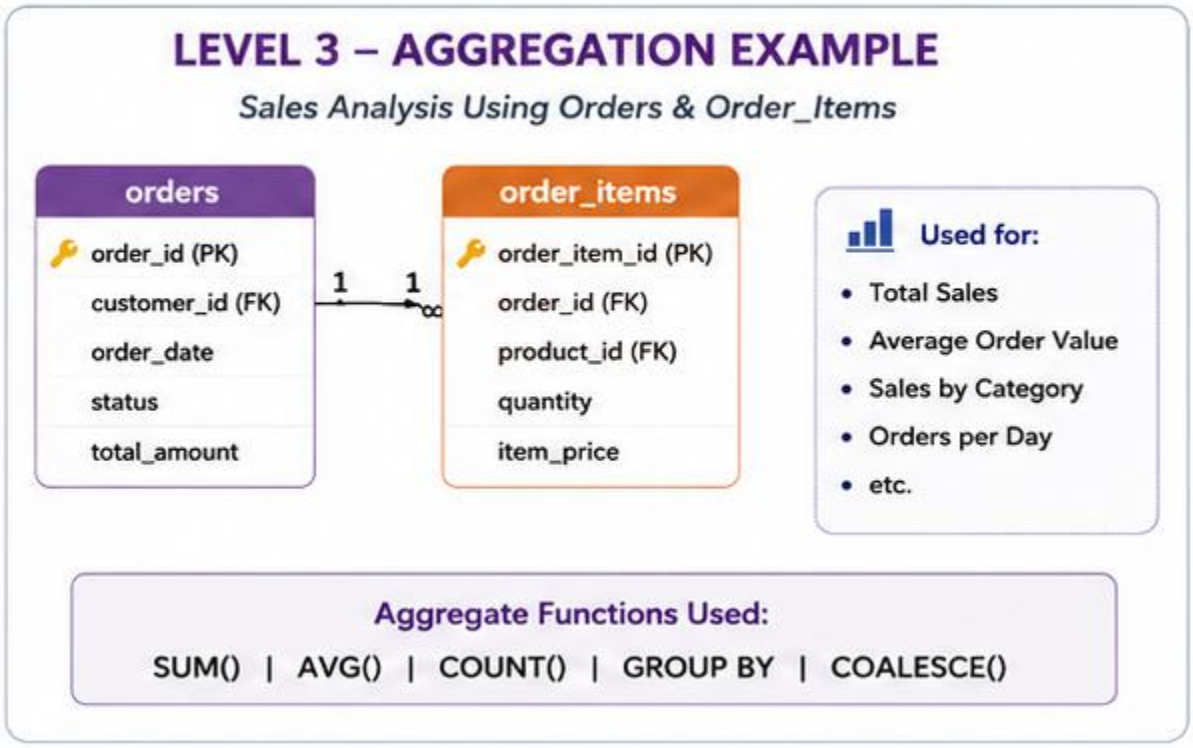
Input: SELECT * FROM products WHERE stock_quantity = 0;

Output:

No products are currently out of stock.

product_id	name	category	price	stock_quantity	added_on
NULL	NULL	NULL	NULL	NULL	NULL

Level 3: Aggregations



Q1. Count the total number of orders placed.

Input: SELECT COUNT(*) as total_orders FROM orders;

Output:

total_orders
400

Q2. Calculate the total revenue collected from all orders.

Input: SELECT SUM(total_amount) as total_revenue FROM orders;

Output:

total_revenue
6960973.66

Q3. Calculate the average order value.

Input: SELECT ROUND(AVG(total_amount),2) as avg_order_value FROM orders;

Output:

avg_order_value
17402.43

Q4. Count the number of customers who have placed at least one order.

Input: SELECT COUNT(DISTINCT customer_id) as total_customers FROM orders;

Output:

total_customers
30

Q5. Find the number of orders placed by each customer.

Input: SELECT DISTINCT customer_id, COUNT(order_id) as total_orders FROM orders GROUP BY customer_id ORDER BY total_orders DESC;

Output:

customer_id	total_orders
29	20
2	17
3	17
17	17
20	17
28	17
6	16
16	16
30	16
14	15
15	15
19	15
22	15
24	15
25	15
5	14
7	13

Q6. Find total sales amount made by each customer.

Input: SELECT customer_id, SUM(total_amount) as total_sales FROM orders GROUP BY customer_id;

Output:

customer_id	total_sales
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78
9	106226.03
10	252722.75
11	173409.01
12	169653.79
13	152727.70
14	360324.18
15	260499.91
16	278469.73
17	262189.71

Q7. List the number of products sold per category.

Input: : SELECT p.category, SUM(oi.quantity) as total_units_sold FROM order_items oi INNER JOIN products p ON p.product_id = oi.product_id GROUP BY p.category;

Output:

category	total_units_sold
Home	443
Clothing	559
Electronics	687
Toys	405
Books	350

Q8. Find the average item price per category.

Input: SELECT p.category, ROUND(AVG(oi.item_price),2) as avg_item_price FROM order_items oi INNER JOIN products p ON p.product_id = oi.product_id GROUP BY p.category;

Output:

category	avg_item_price
Home	2330.71
Clothing	3473.17
Electronics	2733.93
Toys	2407.63
Books	3216.39

Q9. Show number of orders placed per day.

Input: SELECT DISTINCT(day(order_date)) as order_day , COUNT(order_id) as total_orders FROM orders GROUP BY order_date;

Output:

order_day	total_orders
1	1
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	1
13	1
14	1
15	1
16	1
17	1

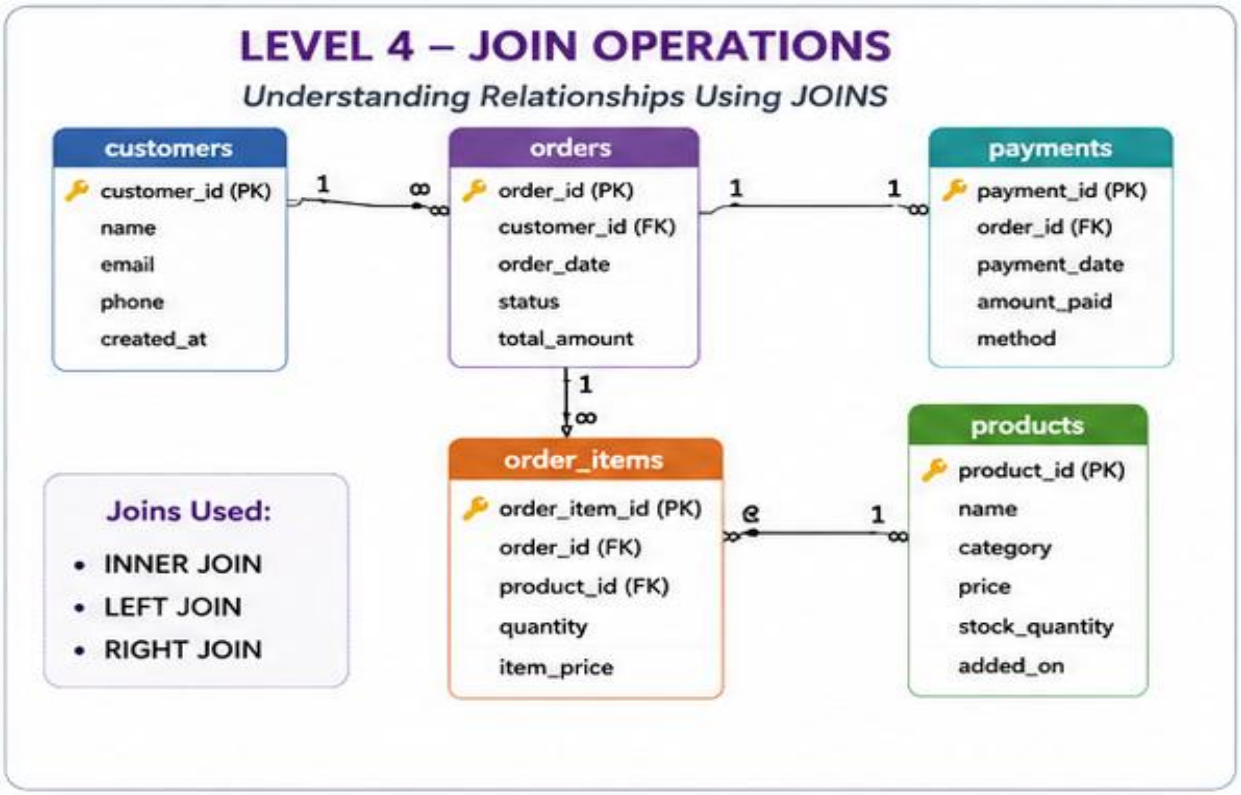
Q10. List total payments received per payment method.

Input: SELECT DISTINCT method, SUM(amount_paid) as total_payment FROM payments GROUP BY method ORDER BY total_payment DESC;

Output:

method	total_payment
Debit Card	1930577.88
Credit Card	1754603.10
Net Banking	1658383.90
UPI	1617408.78

Level 4: Multi-Table Queries (Joins)



Q1. Retrieve order details along with the customer name (INNER JOIN).

Input: SELECT c.name, o.order_id, o.order_date, o.status, o.total_amount FROM customers c INNER JOIN orders o ON c.customer_id = o.customer_id ORDER BY order_id ASC ;

Output:

name	order_id	order_date	status	total_amount
Megan Lee	1	2025-03-02 07:20:11	Delivered	9414.28
Jeffrey Bray	2	2024-10-09 18:08:21	Shipped	532.20
Austin Flores	3	2025-05-08 00:08:27	Cancelled	5164.56
Walter Jenkins	4	2024-09-19 22:16:13	Delivered	9469.78
Mary Knight	5	2025-04-08 18:02:06	Pending	14501.86
Kara Zavala	6	2024-10-25 07:33:59	Cancelled	31050.17
Peter Phillips	7	2024-07-29 11:58:47	Shipped	3043.67
Amanda Bright	8	2024-07-30 22:49:49	Cancelled	32714.06
Mark Duke	9	2025-06-10 17:00:25	Pending	24219.20
Joseph Stuart	10	2025-02-16 12:45:59	Delivered	24342.52
Cindy Hart	11	2025-03-11 14:22:46	Cancelled	16196.13
Joseph Stuart	12	2025-01-15 13:22:53	Shipped	9890.72
Cindy Hart	13	2025-05-12 13:04:29	Cancelled	9627.36
Thomas Owens	14	2024-09-24 21:21:38	Shipped	15803.34
Kara Zavala	15	2024-07-18 03:18:24	Pending	12421.88
David Allen	16	2024-09-09 03:00:40	Shipped	22856.73
Thomas Owens	17	2024-08-19 21:17:57	Pending	11173.77

Q2. Get list of products that have been sold (INNER JOIN with order_items).

Input: SELECT DISTINCT p.product_id,p.name, p.category FROM products p INNER JOIN order_items oi ON p.product_id = oi.product_id;

Output:

product_id	name	category
1	Plant No	Home
2	Population Social	Clothing
3	Available Answer	Electronics
4	Any Question	Clothing
5	Natural Network	Toys
6	If Whatever	Electronics
7	Response Indeed	Clothing
8	Every Amount	Home
9	Common Study	Toys
10	Development System	Electronics
11	Build Her	Books
12	Action Ask	Electronics
13	Full West	Books
14	Listen Development	Home
15	Place Low	Electronics
16	Special Fact	Toys
17	Everything Plant	Books

Q3. List all orders with their payment method (INNER JOIN) .

Input: SELECT o.*, p.method as payment_method FROM payments p INNER JOIN orders o ON o.order_id = p.order_id;

Output:

order_id	customer_id	order_date	status	total_amount	payment_method
1	20	2025-03-02 07:20:11	Delivered	9414.28	Credit Card
2	18	2024-10-09 18:08:21	Shipped	532.20	Net Banking
3	15	2025-05-08 00:08:27	Cancelled	5164.56	Credit Card
4	11	2024-09-19 22:16:13	Delivered	9469.78	UPI
5	12	2025-04-08 18:02:06	Pending	14501.86	UPI
6	29	2024-10-25 07:33:59	Cancelled	31050.17	UPI
7	22	2024-07-29 11:58:47	Shipped	3043.67	UPI
8	19	2024-07-30 22:49:49	Cancelled	32714.06	Net Banking
9	6	2025-06-10 17:00:25	Pending	24219.20	Net Banking
10	28	2025-02-16 12:45:59	Delivered	24342.52	Debit Card
11	25	2025-03-11 14:22:46	Cancelled	16196.13	Credit Card
12	28	2025-01-15 13:22:53	Shipped	9890.72	Credit Card
13	25	2025-05-12 13:04:29	Cancelled	9627.36	UPI
14	1	2024-09-24 21:21:38	Shipped	15803.34	UPI
15	29	2024-07-18 03:18:24	Pending	12421.88	UPI
16	5	2024-09-09 03:00:40	Shipped	22856.73	UPI
17	1	2024-08-19 21:17:57	Pending	11173.77	UPI

Q4. Get list of customers and their orders (LEFT JOIN).

Input: SELECT c.name, c.customer_id, o.order_id. o.order_date, o.status, o.total_amount FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;

Output:

name	customer_id	order_id	order_date	status	total_amount
Thomas Owens	1	14	2024-09-24 21:21:38	Shipped	15803.34
Thomas Owens	1	17	2024-08-19 21:17:57	Pending	11173.77
Thomas Owens	1	61	2024-12-26 23:21:58	Shipped	13053.00
Thomas Owens	1	76	2025-01-14 22:59:54	Pending	23506.81
Thomas Owens	1	92	2024-09-26 11:33:58	Shipped	834.75
Thomas Owens	1	109	2025-03-17 04:56:18	Shipped	12190.14
Thomas Owens	1	127	2025-06-10 18:01:32	Pending	20160.64
Thomas Owens	1	135	2025-03-11 22:58:23	Pending	8891.79
Thomas Owens	1	144	2024-09-19 09:18:22	Shipped	12093.54
Thomas Owens	1	221	2024-09-25 05:49:39	Pending	12437.61
Thomas Owens	1	278	2025-02-01 20:05:38	Delivered	32015.16
Thomas Owens	1	379	2025-03-22 15:57:58	Cancelled	21586.89
Charles Grant	2	56	2024-06-23 15:59:57	Pending	6828.61
Charles Grant	2	169	2025-02-11 04:04:38	Cancelled	9426.30
Charles Grant	2	209	2024-10-16 10:07:48	Shipped	7690.62
Charles Grant	2	223	2024-11-25 01:32:18	Cancelled	41020.75

Q5. List all products along with order item quantity (LEFT JOIN).

Input: SELECT p.product_id, p.name, p.category, COALESCE(SUM(oi.quantity),0) as total_units_sold FROM products p LEFT JOIN order_items oi ON p.product_id = oi.product_id GROUP BY p.product_id, p.name, p.category ORDER BY total_units_sold DESC;

Output:

product_id	name	category	total_units_sold
24	Assume Serve	Home	83
46	Age Treatment	Home	67
18	Some Them	Toys	63
47	Southern Thing	Electronics	62
8	Every Amount	Home	61
22	Move Small	Books	60
20	Real Source	Books	60
37	Between Up	Toys	58
17	Everything Plant	Books	58
48	Group But	Clothing	58
35	Fire Often	Electronics	58
9	Common Study	Toys	57
31	Series Page	Electronics	54
6	If Whatever	Electronics	54
43	Least Green	Toys	54
10	Development S...	Electronics	53
50	Stock Article	Home	53

Q6. List all payments including those with no matching orders (RIGHT JOIN).

Input: SELECT p.payment_id, o.order_date, o.status, p.payment_date, o.total_amount, p.method FROM orders o RIGHT JOIN payments p ON o.order_id = p.order_id ;

Output:

payment_id	order_date	status	payment_date	total_amount	method
1	2025-03-02 07:20:11	Delivered	2025-03-02 08:16:11	9414.28	Credit Card
2	2024-10-09 18:08:21	Shipped	2024-10-09 19:00:21	532.20	Net Banking
3	2025-05-08 00:08:27	Cancelled	2025-05-08 00:55:27	5164.56	Credit Card
4	2024-09-19 22:16:13	Delivered	2024-09-19 22:28:13	9469.78	UPI
5	2025-04-08 18:02:06	Pending	2025-04-08 18:26:06	14501.86	UPI
6	2024-10-25 07:33:59	Cancelled	2024-10-25 08:16:59	31050.17	UPI
7	2024-07-29 11:58:47	Shipped	2024-07-29 12:45:47	3043.67	UPI
8	2024-07-30 22:49:49	Cancelled	2024-07-30 23:42:49	32714.06	Net Banking
9	2025-06-10 17:00:25	Pending	2025-06-10 17:36:25	24219.20	Net Banking
10	2025-02-16 12:45:59	Delivered	2025-02-16 13:43:59	24342.52	Debit Card
11	2025-03-11 14:22:46	Cancelled	2025-03-11 15:05:46	16196.13	Credit Card
12	2025-01-15 13:22:53	Shipped	2025-01-15 14:00:53	9890.72	Credit Card
13	2025-05-12 13:04:29	Cancelled	2025-05-12 14:01:29	9627.36	UPI
14	2024-09-24 21:21:38	Shipped	2024-09-24 22:11:38	15803.34	UPI
15	2024-07-18 03:18:24	Pending	2024-07-18 03:45:24	12421.88	UPI
16	2024-09-09 03:00:40	Shipped	2024-09-09 03:38:40	22856.73	UPI
17	2024-08-19 21:17:57	Pending	2024-08-19 21:49:57	11173.77	UPI

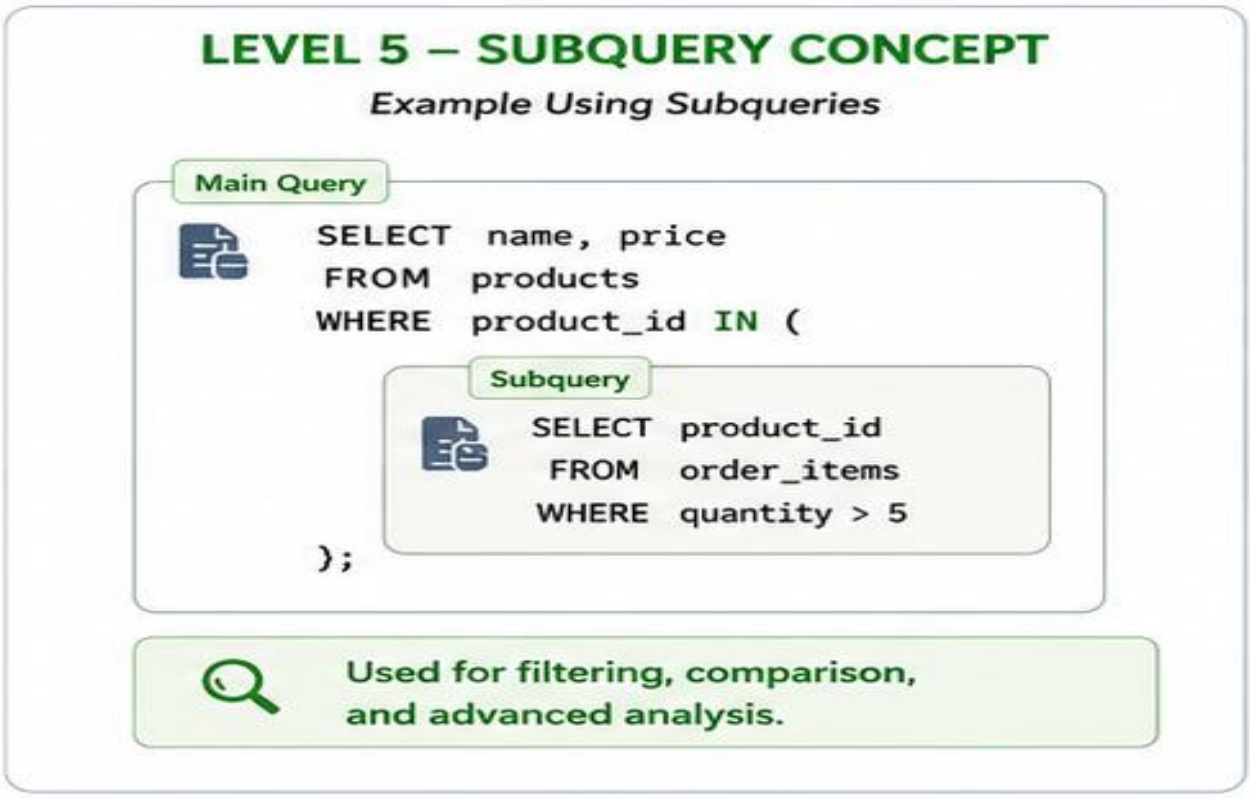
Q7. Combine data from three tables: customer, order, and payment .

Input: SELECT c.customer_id, c.name, c.email, c.phone, o.order_id, o.order_date, o.status, o.total_amount, p.payment_date, p.amount_paid, p.method FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id LEFT JOIN payments p On p.order_id = o.order_id ORDER BY order_id ASC;

Output:

customer_id	name	email	phone	order_id	order_date	status	total_amount	payment_date	amount_paid	method
20	Megan Lee	user20@example.com	7044478333	1	2025-03-02 07:20:11	Delivered	9414.28	2025-03-02 08:16:11	9414.28	Credit Card
18	Jeffrey Bray	user18@example.com	164.962.0222x92	2	2024-10-09 18:08:21	Shipped	532.20	2024-10-09 19:00:21	532.20	Net Banking
15	Austin Flores	user15@example.com	329.901.1576x66	3	2025-05-08 00:08:27	Cancelled	5164.56	2025-05-08 00:55:27	5164.56	Credit Card
11	Walter Jenkins	user11@example.com	536-329-0817x71	4	2024-09-19 22:16:13	Delivered	9469.78	2024-09-19 22:28:13	9469.78	UPI
12	Mary Knight	user12@example.com	361-636-3802	5	2025-04-08 18:02:06	Pending	14501.86	2025-04-08 18:26:06	14501.86	UPI
29	Kara Zavala	user29@example.com	434-985-5776x61	6	2024-10-25 07:33:59	Cancelled	31050.17	2024-10-25 08:16:59	31050.17	UPI
22	Peter Phillips	user22@example.com	389-656-1695	7	2024-07-29 11:58:47	Shipped	3043.67	2024-07-29 12:45:47	3043.67	UPI
19	Amanda Bright	user19@example.com	380.981.9798x69	8	2024-07-30 22:49:49	Cancelled	32714.06	2024-07-30 23:42:49	32714.06	Net Banking
6	Mark Duke	user6@example.com	(144)957-2811	9	2025-06-10 17:00:25	Pending	24219.20	2025-06-10 17:36:25	24219.20	Net Banking
28	Joseph Stuart	user28@example.com	363-045-4287	10	2025-02-16 12:45:59	Delivered	24342.52	2025-02-16 13:43:59	24342.52	Debit Card
25	Cindy Hart	user25@example.com	617.574.8421x41	11	2025-03-11 14:22:46	Cancelled	16196.13	2025-03-11 15:05:46	16196.13	Credit Card
28	Joseph Stuart	user28@example.com	363-045-4287	12	2025-01-15 13:22:53	Shipped	9890.72	2025-01-15 14:00:53	9890.72	Credit Card
25	Cindy Hart	user25@example.com	617.574.8421x41	13	2025-05-12 13:04:29	Cancelled	9627.36	2025-05-12 14:01:29	9627.36	UPI
1	Thomas Owens	user1@example.com	142-479-1945	14	2024-09-24 21:21:38	Shipped	15803.34	2024-09-24 22:11:38	15803.34	UPI
29	Kara Zavala	user29@example.com	434-985-5776x61	15	2024-07-18 03:18:24	Pending	12421.88	2024-07-18 03:45:24	12421.88	UPI
5	David Allen	user5@example.com	(751)456-8289x1	16	2024-09-09 03:00:40	Shipped	22856.73	2024-09-09 03:38:40	22856.73	UPI
1	Thomas Owens	user1@example.com	142-479-1945	17	2024-08-19 21:17:57	Pending	11173.77	2024-08-19 21:49:57	11173.77	UPI

Level 5: Subqueries (Inner Queries)



Q1. List all products priced above the average product price.

Input: SELECT name, price FROM products WHERE price > (SELECT AVG(price) as avg_price FROM products);

Output:

name	price
Population Social	4813.68
Any Question	4759.28
Natural Network	4722.66
Response Indeed	4897.36
Every Amount	4173.60
Development System	4801.78
Action Ask	4017.01
Some Them	3673.86
Build High	4707.14
Real Source	4398.66
Bed Including	3845.31
Move Small	4168.08
Assume Serve	3437.81
South Free	3543.58
Movement Future	3502.62
Drop Remember	4160.45
Future Sort	3043.67

Q2. Find customers who have placed at least one order.

Input: SELECT customer_id , name FROM customers WHERE customer_id IN (SELECT DISTINCT(customer_id) FROM orders);

Output:

customer_id	name
1	Thomas Owens
2	Charles Grant
3	Kaitlin Richards
4	Christina Williams
5	David Allen
6	Mark Duke
7	Briana Wright
8	John Bryan
9	Jason Thompson
10	Shawn Hill
11	Walter Jenkins
12	Mary Knight
13	Leslie Wilson
14	Deborah Arias
15	Austin Flores
16	Amy Landry
17	Randy Mooney

Q3. Show orders whose total amount is above the average for that customer.

Input: SELECT order_id, customer_id, total_amount FROM orders o WHERE total_amount > (SELECT AVG(total_amount) FROM orders WHERE customer_id = o.customer_id) ORDER BY customer_id, total_amount DESC;

Output:

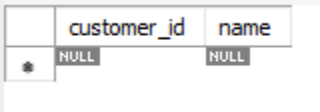
order_id	customer_id	total_amount
278	1	32015.16
76	1	23506.81
379	1	21586.89
127	1	20160.64
14	1	15803.34
315	2	42056.04
223	2	41020.75
329	2	31387.01
334	2	27857.86
230	2	22655.32
394	2	18154.30
281	3	41679.11
63	3	36236.03
369	3	33503.41
42	3	21622.22
234	3	18473.60
134	3	15483.53

Q4. Display customers who haven't placed any orders.

Input: SELECT customer_id, name FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

Output:

No customers without orders were found.



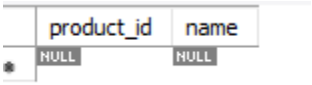
	customer_id	name
*	NULL	NULL

Q5. Show products that were never ordered.

Input: SELECT p.product_id, p.name FROM products p WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);

Output:

No products without orders were found.



	product_id	name
*	NULL	NULL

Q6. Show highest value order per customer.

Input: SELECT order_id, customer_id, total_amount FROM orders o WHERE total_amount = (SELECT MAX(total_amount) FROM orders WHERE customer_id = o.customer_id) ORDER BY customer_id ASC;

Output:

order_id	customer_id	total_amount
278	1	32015.16
315	2	42056.04
281	3	41679.11
386	4	25747.34
266	5	39921.78
330	6	39003.19
128	7	28589.04
33	8	36147.09
348	9	21414.44
62	10	35723.17
133	11	37129.82
80	12	38656.26
18	13	32001.24
107	14	48042.06
243	15	40510.54
121	16	37415.67
256	17	40944.10

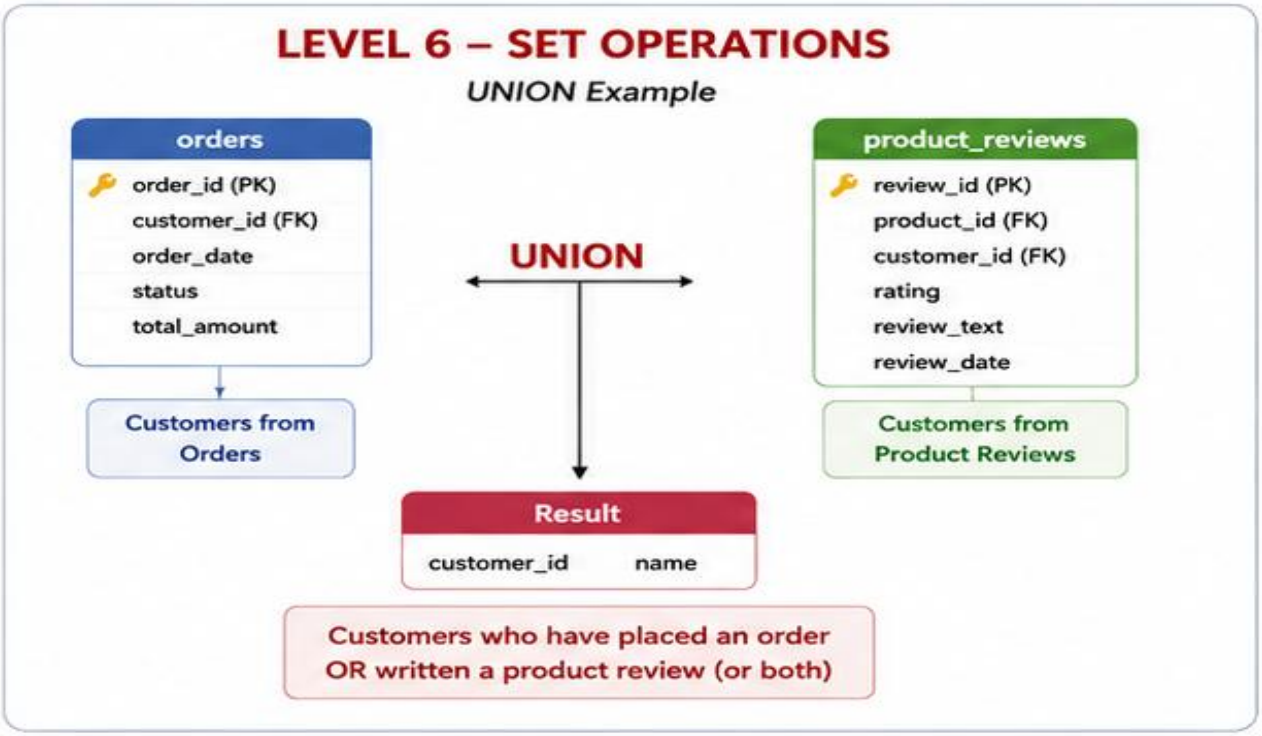
Q7. Highest Order Per Customer (Including Names).

Input: SELECT (SELECT name FROM customers WHERE customer_id = o.customer_id) as customer_name, order_id, customer_id, total_amount FROM orders o WHERE total_amount = (SELECT MAX(total_amount) FROM orders WHERE customer_id = o.customer_id) ORDER BY customer_id ASC;

Output:

customer_name	order_id	customer_id	total_amount
Thomas Owens	278	1	32015.16
Charles Grant	315	2	42056.04
Kaitlin Richards	281	3	41679.11
Christina Williams	386	4	25747.34
David Allen	266	5	39921.78
Mark Duke	330	6	39003.19
Briana Wright	128	7	28589.04
John Bryan	33	8	36147.09
Jason Thompson	348	9	21414.44
Shawn Hill	62	10	35723.17
Walter Jenkins	133	11	37129.82
Mary Knight	80	12	38656.26
Leslie Wilson	18	13	32001.24
Deborah Arias	107	14	48042.06
Austin Flores	243	15	40510.54
Amy Landry	121	16	37415.67
Randy Mooney	256	17	40944.10

Level 6: Set Operations



Q1. List all customers who have either placed an order or written a product review.

Input: SELECT name, customer_id FROM customers WHERE customer_id IN (SELECT customer_id FROM orders UNION SELECT customer_id FROM product_reviews);

Output:

name	customer_id
Thomas Owens	1
Charles Grant	2
Kaitlin Richards	3
Christina Williams	4
David Allen	5
Mark Duke	6
Briana Wright	7
John Bryan	8
Jason Thompson	9
Shawn Hill	10
Walter Jenkins	11
Mary Knight	12
Leslie Wilson	13
Deborah Arias	14
Austin Flores	15
Amy Landry	16
Randy Mooney	17

Q2. List all customers who have placed an order as well as reviewed a product [Intersect not supported].

Input: SELECT name, customer_id FROM customers WHERE customer_id IN (SELECT customer_id FROM orders)
AND customer_id IN (SELECT customer_id FROM product_reviews);

Output:

name	customer_id
Thomas Owens	1
Charles Grant	2
Christina Williams	4
Mark Duke	6
Briana Wright	7
Jason Thompson	9
Shawn Hill	10
Walter Jenkins	11
Leslie Wilson	13
Deborah Arias	14
Austin Flores	15
Randy Mooney	17
Amanda Bright	19
Megan Lee	20
Jessica Zamora	21
Peter Phillips	22
Kathryn Mathews	23

PROJECT SUMMARY

This project analyzes a retail store database using MySQL to retrieve and analyze information from customers, products, orders, order items, payments, and product reviews. The project covers basic SQL queries, filtering, aggregate functions, grouping, multi-table joins, subqueries, correlated subqueries, and set operations. These queries help generate useful insights such as customer orders, product sales, payment information, average order value, highest-value orders, and customer activity. The project demonstrates how MySQL can be used to efficiently manage, retrieve, and analyze data in a relational database.

SQL CONCEPTS USED

SELECT	Retrieving data
WHERE	Filtering records
GROUP BY	Grouping data
SUM()	Calculating sales
AVG()	Calculating averages
COUNT()	Counting records
INNER JOIN	Combining related tables
LEFT JOIN	Keeping unmatched records
SUBQUERY	Comparing against calculated values
UNION	Combining result sets
DISTINCT	Removing duplicate results
ORDER BY	Sorting records
MAX()	Finding highest values
RIGHT JOIN	Including unmatched payment records
NOT EXISTS	Finding records with no matching records
CORRELATED SUBQUERY	Customer-specific comparisons
COALESCE()	Replacing NULL values with 0

Conclusion

This project provided practical experience in using MySQL to manage, retrieve, and analyze data from a relational retail store database.

Key Learnings

- **Understanding relational databases and table relationships**
- **Writing basic and advanced SQL queries**
- **Using aggregate functions and grouping for analysis**
- **Applying different types of Joins**
- **Using subqueries and Correlated Subqueries**
- **Applying set operations such as UNION**
- **Extracting meaningful business insights from retail data.**